ITA0613 - Machine Learning

Assignment Questions

NAME : GODUGULA GOUTHAM

REG NO : 192312187

Course Code & Name	ITA0613 - Machine Learning
Assignment Title	Large-Scale Instance-Based and Statistical Learning Pipeline for Climate-Resilient Crop Yield Forecasting
Course Outcome(s) - CO	CO2: Provide an overview of various learning paradigms and strategies, focusing on decision tree learning and concept representation (version spaces, inductive bias). CO6: Examine instance-based learning approaches including k-nearest neighbors, locally weighted regression, and case-based reasoning. CO7: Master data manipulation and analysis skills using tools like NumPy and Pandas for statistical analysis, aggregation, and visualization.
Bloom's Taxonomy Level	L5 - Evaluate; L6 - Create
SDG Mapping (SDG 1-17)	SDG 2 - Zero Hunger; SDG 13 - Climate Action

Problem Statement

An agricultural research consortium wants to forecast regional crop yields under increasingly variable climate conditions, so that farmers and policymakers can make climate-resilient planting decisions - directly supporting SDG 2 - Zero Hunger and SDG 13 - Climate Action. You are given (or must responsibly source) a multi-year agro-climatic dataset containing historical weather variables (rainfall, temperature, humidity), soil parameters, and recorded crop yields across multiple districts and seasons. As the lead data scientist, design, implement, and rigorously evaluate a complete instance-based and statistical learning pipeline, without using any pre-built machine-learning library for the core modelling.

(a) Perform a full exploratory data analysis using only NumPy and Pandas - cleaning the data, imputing missing weather records, merging multiple yearly/regional files, and engineering at least three derived features (for example, growing-degree days or a rainfall-anomaly index).

(b) Implement a k-Nearest Neighbour regressor entirely from first principles (no scikit-learn) to predict yield for an unseen district-season combination, including your own implementation of at least two distance metrics (e.g., Euclidean and Mahalanobis) and a justified choice of k based on a validation curve you compute manually.

(c) Implement Locally Weighted Regression from first principles, deriving and applying the weighting kernel, and compare its bias-variance behaviour against the k-NN regressor both theoretically and empirically.

(d) Apply the Candidate-Elimination algorithm (or an adapted version-space approach) to a discretised version of the problem (e.g., classifying yield into low/medium/high risk bands) to identify the boundary hypotheses consistent with historical seasons, and explain the inductive bias of your chosen representation.

(e) Analyse computational feasibility: measure and report the time and memory complexity of your k-NN and locally-weighted-regression implementations as the dataset scales from 10^3 to 10^6 records, and propose and prototype at least one indexing or approximation strategy (e.g., a k-d tree or an approximate-nearest-neighbour hashing scheme) to make the pipeline scalable.

(f) Produce at least five distinct, publication-quality visualisations (using Pandas/Matplotlib) that reveal actionable climate-yield insights, and write a concise, data-driven policy brief interpreting the results for a non-technical agricultural-policy audience.

(g) Critically evaluate the limitations, uncertainty, and fairness implications of forecasting future yield from historical data under a changing climate, and explain how the solution advances SDG 2 and SDG 13 targets.

Constraints: All core algorithms (k-NN regression, locally weighted regression, candidate-elimination/version-space reasoning) must be implemented from first principles using only NumPy/Pandas - no scikit-learn or other pre-built ML library may be used for the core modelling. Use a real, publicly available agro-climatic dataset with at least 10,000 records spanning multiple years and regions. The complete pipeline must run end-to-end on the full dataset without manual intervention, within a time budget that you define and justify.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Data Engineering & Exploratory Analysis (CO7)	15	Thorough cleaning, imputation, and merging; at least three well-justified engineered features using only NumPy/Pandas.	Good data preparation with minor gaps in feature engineering.	Basic cleaning/merging done; feature engineering shallow or partial.	Data preparation missing, incorrect, or reliant on disallowed libraries.
k-NN Regressor Implementation & Distance Metrics (CO6)	20	k-NN implemented from scratch with two correctly implemented distance metrics and a well-justified k via a manually computed validation curve.	Correct k-NN with minor gaps in metric implementation or k-selection justification.	k-NN works but only one distance metric or weak k-justification.	k-NN missing, incorrect, or built using a pre-built library.
Locally Weighted Regression & Bias-Variance Comparison (CO6)	15	Correct from-scratch implementation with a clear, correct theoretical and empirical bias-variance comparison to k-NN.	Correct implementation; comparison present but not fully rigorous.	Implementation works but comparison is superficial or partly incorrect.	LWR missing, incorrect, or comparison absent.
Version-Space / Candidate-Elimination Analysis (CO1/CO2)	15	Correct application of Candidate-Elimination to the discretised	Mostly correct application with minor gaps in the bias	Basic application with incomplete or partly incorrect boundary	Not attempted or fundamentally incorrect.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
		problem with a clear, well-justified discussion of inductive bias.	discussion.	hypotheses.	
Scalability Analysis & Optimisation Prototype	15	Rigorous complexity measurement across scales with a working, clearly beneficial indexing/approximation prototype.	Reasonable measurement and a working prototype with modest gains.	Measurement attempted but prototype incomplete or gains unclear.	No scalability analysis or optimisation attempted.
Visualisation, Policy Brief & SDG 2/13 Relevance	10	Five or more insightful, publication-quality visualisations with a clear, well-argued policy brief tied to SDG 2/13.	Good visualisations and brief with minor gaps in insight or SDG linkage.	Visualisations/brief present but generic or weakly connected to SDGs.	Visualisations or brief missing or uninformative.
GitHub Repository Quality, Documentation & CI	10	Clean modular repository, incremental commit history, complete README, automated tests, and a working CI pipeline.	Well-organised repository with most of the above present.	Repository present but poorly organised or missing tests/CI.	Repository missing, disorganised, or submitted as a single non-incremental upload.

Deliverables

To receive full credit, each submission must include the following GitHub-based deliverables (this is a repository-graded assignment - no offline/zip submissions accepted):

1. A complete, modular GitHub repository containing all source code (data pipeline, k-NN, locally weighted regression, candidate-elimination module, and the scalability-optimisation prototype), built through an incremental commit history that demonstrates genuine development progress (a single 'final upload' commit is not acceptable).
2. A comprehensive README.md documenting the pipeline architecture, dataset source and licensing, setup/run instructions, and the exact steps needed to reproduce every reported result and visualisation.
3. Automated unit tests (e.g., using pytest) covering the core algorithmic components (distance-metric functions, the LWR kernel, the candidate-elimination boundary updates), committed to the repository.
4. A technical report (Markdown or PDF, committed to the repository) containing the mathematical derivations, complexity analysis, scalability results, and the non-technical policy brief.
5. A /results folder in the repository containing all logs, plots, validation curves, and result tables, so every reported figure is independently reproducible from the repository alone.
6. A working GitHub Actions CI workflow (or equivalent) that automatically runs the unit tests on every push, with a passing-build badge displayed in the README.

ABSTRACT

Agricultural yield forecasting is increasingly important because crop productivity is affected by rainfall variability, temperature extremes, soil conditions, and farm inputs. This report presents an end-to-end instance-based and statistical learning pipeline for climate-resilient crop-yield forecasting. The work follows the seven required tasks of the ITA0613 assignment: exploratory data analysis and feature engineering; first-principles k-Nearest Neighbour regression; first-principles Locally Weighted Regression; an adapted Candidate-Elimination/version-space analysis for yield-risk classes; computational scalability analysis and indexing; publication-quality visualisation and policy interpretation; and a critical discussion of uncertainty, limitations, fairness, and SDG relevance.

The implementation deliberately avoids scikit-learn and other pre-built machine-learning libraries for the core models. NumPy and Pandas are used for numerical computation and data preparation, while Matplotlib is used for visualisation. The benchmark used during development contains 20,000 records spanning multiple years, states, crops, and seasons. It includes intentionally missing climate/soil observations to exercise the imputation and preprocessing pipeline. The chronological split produces 16,743 training records, 1,583 validation records, and 1,674 test records. Manual validation selected k=21 for Mahalanobis kNN and a Gaussian LWR bandwidth (tau) of 2.0. On the development benchmark, kNN achieved a test RMSE of 1.8182 and LWR achieved 1.8201.

An important reproducibility qualification is required: the included benchmark is synthetic-but-realistic and was created because the supplied assignment document did not contain the required public dataset. It is therefore a development benchmark, not a substitute for the assignment's requirement to use a real public agro-climatic dataset. Before final submission, it must be replaced by a qualifying public dataset while retaining the same pipeline structure and documenting the source and licence. The report nevertheless demonstrates the complete computational workflow, algorithmic derivations, evaluation methodology, repository organisation, testing, and policy interpretation expected by the brief.

Keywords

Crop yield forecasting; kNN regression; Mahalanobis distance; locally weighted regression; version space; Candidate-Elimination; climate resilience; scalability; NumPy; Pandas; SDG 2; SDG 13.

TABLE OF CONTENTS

1. Introduction and Assignment Alignment

2. Problem Definition and Objectives

3. System Architecture and End-to-End Workflow

4. Task A – Data Engineering and Exploratory Data Analysis

5. Task B – First-Principles k-Nearest Neighbour Regression

6. Task C – Locally Weighted Regression and Bias–Variance Analysis

7. Task D – Candidate-Elimination and Version-Space Analysis

8. Task E – Scalability, Complexity and Indexing Prototype

9. Task F – Visualisation and Agricultural Policy Brief

10. Task G – Limitations, Uncertainty, Fairness and SDGs

11. Software Engineering, Testing, CI and Reproducibility

12. Results and Discussion

13. Conclusion and Future Work

Appendix A – Key Mathematical Formulas

Appendix B – Repository Structure and Reproduction Steps

1. INTRODUCTION AND ASSIGNMENT ALIGNMENT

The supplied ITA0613 assignment defines a large-scale learning problem in which historical agro-climatic observations are used to forecast crop yield under increasingly variable climate conditions. The brief explicitly asks for learning paradigms related to instance-based learning, statistical learning, concept representation and version spaces, together with data manipulation using NumPy and Pandas. It also maps the work to SDG 2 and SDG 13. The required tasks are not independent exercises: they are intended to form a complete engineering pipeline in which data preparation feeds the learning algorithms, model outputs support comparison and policy interpretation, and scalability/testing determine whether the approach is operationally credible.

Assignment element	**Report implementation**	**Evidence / outcome**
Task A	Cleaning, imputation, merging-ready pipeline, 4 derived climate features	20,000-row benchmark; chronological split; standardisation
Task B	kNN from scratch with Euclidean and Mahalanobis distances	Manual validation selected k=21; test RMSE 1.8182
Task C	Gaussian-kernel LWR from scratch	Manual bandwidth selection tau=2.0; test RMSE 1.8201
Task D	Discretised yield-risk classes with adapted version-space reasoning	Low/Medium/High boundaries and inductive-bias analysis
Task E	Scaling experiment and exact k-d tree prototype	10^3 to 10^6 benchmark design and indexing prototype
Task F	Matplotlib visualisations and policy brief	Eight visualisation outputs plus non-technical recommendations
Task G	Limitations, uncertainty, fairness, SDG interpretation	Climate-shift and distributional-risk discussion

The assignment also requires a modular GitHub repository, incremental development history, a comprehensive README, automated tests, a technical report, a /results directory, and a GitHub Actions workflow. The implementation was organised accordingly so that the final repository can be evaluated as a software project rather than only as a notebook-style analysis.

1.1 ALIGNMENT WITH COURSE OUTCOMES

- CO2: The report applies concept representation and version-space reasoning to discretised yield-risk classes and explicitly discusses inductive bias.
- CO6: The central predictive methods are kNN and locally weighted regression, implemented from first principles.
- CO7: NumPy and Pandas provide data manipulation, aggregation, transformation, statistical calculations and model-supporting preprocessing.

1.2 BLOOM'S TAXONOMY AND SDG CONTEXT

The assignment is positioned at L5 (Evaluate) and L6 (Create). Accordingly, the report does not stop at implementing algorithms; it evaluates model behaviour, computational cost, uncertainty and practical relevance. SDG 2 is addressed through improved crop-yield information for food-security planning, while SDG 13 is addressed by explicitly modelling climate-sensitive variables and discussing how historical relationships can fail under future climate regimes.

2. PROBLEM DEFINITION AND OBJECTIVES

2.1 PROBLEM STATEMENT

The agricultural forecasting problem can be expressed as follows. Given a set of historical records describing region, season, crop, weather conditions, soil characteristics and management inputs, estimate the yield y for an unseen district/region-season combination. The predictor must operate without a pre-built machine-learning library for the core learning algorithms. It must also expose the relationship between local similarity, statistical weighting, computational cost and uncertainty.

2.2 OBJECTIVES

1. Construct a robust data-processing pipeline using NumPy and Pandas.
2. Clean missing values and derive climate-sensitive explanatory variables without target leakage.
3. Implement kNN regression from first principles using at least two distance metrics.
4. Select k through a manually computed validation curve rather than a library optimiser.
5. Implement locally weighted regression with a mathematically defined kernel and bandwidth selection.
6. Compare the theoretical and empirical bias–variance behaviour of kNN and LWR.
7. Adapt Candidate-Elimination/version-space reasoning to Low/Medium/High yield-risk classes.
8. Measure computational scaling and prototype an indexing strategy using a k-d tree.
9. Produce reproducible visualisations and a policy brief suitable for non-technical decision makers.
10. Document limitations, uncertainty and fairness under climate distribution shift.

2.3 DESIGN CONSTRAINTS

- Core algorithms are implemented from scratch.
- NumPy/Pandas are used for numerical and tabular processing.
- No scikit-learn is used for kNN, LWR, version-space reasoning or model selection.
- The final submission must use a real public dataset with at least 10,000 records spanning multiple years and regions.
- The pipeline must run end-to-end without manual intervention and report a justified time budget.
- The GitHub repository must show incremental development, tests and CI.

3. SYSTEM ARCHITECTURE AND END-TO-END WORKFLOW

The pipeline is deliberately modular. Task A creates the clean feature matrix and chronological train/validation/test partitions. Task B consumes the training and validation matrices to tune k and produces test predictions. Task C uses the same partitions to tune its bandwidth and produces a second set of predictions. Task D transforms yield into risk classes and applies adapted version-space reasoning. Task E evaluates how neighbour search behaves as the record count increases and compares brute-force search with a k-d tree. Task F consumes the processed data and model results to generate figures and a policy brief. Task G audits model errors by region and crop and synthesises limitations, uncertainty and fairness implications.

Module	Main responsibility	Inputs	Outputs
main.py	Orchestrator	All task modules	A→G completion and summary
task_a.py	Data engineering	Raw CSV	Clean data, splits, metadata
task_b.py	kNN	A outputs	Validation curve, predictions, metrics
task_c.py	LWR	A outputs	Bandwidth curve, predictions, metrics
task_d.py	Version space	A target/risk labels	Boundary hypotheses
task_e.py	Scalability	Feature matrix	Timing/memory/index results
task_f.py	Visualisation	A/B/C/E outputs	PNG figures and policy brief
task_g.py	Critical evaluation	Predictions + metadata	Fairness/uncertainty report

3.1 DATA FLOW

Raw records → validation and schema checks → missing-value imputation → feature engineering → chronological splitting → numerical standardisation → kNN/LWR/version-space/scalability modules → metrics and plots → policy and critical evaluation.

3.2 LEAKAGE PREVENTION

Production and area are excluded from the predictor set because production is directly related to yield and can introduce target leakage depending on the data-generating process. The development feature matrix therefore concentrates on climate, soil and management variables plus derived climate indicators. Chronological splitting also prevents future observations from influencing the training set when evaluating historical-to-future forecasting behaviour.

4. TASK A – DATA ENGINEERING AND EXPLORATORY DATA ANALYSIS

4.1 DATASET AND REPRODUCIBILITY STATUS

The development run used a 20,000-row synthetic-but-realistic benchmark spanning 1997–2020, multiple Indian states, six crop categories and four seasons. Missing weather and soil values were deliberately inserted so that the pipeline could demonstrate its imputation logic. This benchmark is useful for testing the software, but it does not satisfy the assignment's final requirement for a real public dataset. The final submission must replace it with a qualifying public dataset and document the source and licence in the README and report.

Property	Development benchmark
Rows	20,000
Years	1997–2020
Regions	10 Indian states
Crops	6
Seasons	4
Training records	16,743
Validation records	1,583
Test records	1,674
Missing values	Intentionally injected in climate/soil fields
Target	Crop yield

4.2 CLEANING AND IMPUTATION

Data cleaning checks include duplicate detection, data-type coercion, invalid numeric values, missingness summaries, and chronological ordering. For the development pipeline, numeric climate/soil variables are imputed using state-level medians, which is preferable to a single global median when regional climate regimes differ. The imputation statistics are calculated from the training portion before being applied to validation and test data to avoid using future information.

4.3 DERIVED FEATURES

Derived feature	Definition / rationale
Growing Degree Days (GDD)	Approximate accumulated thermal time above a crop-relevant base temperature.
Rainfall anomaly	Difference or normalised deviation of rainfall from a historical regional baseline.
Temperature range	Maximum temperature minus minimum temperature; captures diurnal thermal variability.
Heat stress indicator	A measure of exposure to temperatures above a chosen stress threshold.

These features convert raw climate observations into signals more closely connected to crop physiology and climate stress. The exact base temperature and stress threshold should be recalibrated for crop-specific scientific use rather than treated as universal constants.

4.4 EXPLORATORY FINDINGS

- Yield distributions differ across crop and regional groups, motivating group-aware error analysis.

- Rainfall and temperature variability introduce nonlinear and local effects that motivate instance-based methods.
- Climate-derived variables provide additional context beyond raw weather measurements.
- Regional imputation is preferable to indiscriminate global imputation because climate distributions are spatially heterogeneous.
- Chronological evaluation is more realistic than a random split for forecasting future seasons.

4.5 STANDARDISATION

Each continuous predictor is standardised using training-set statistics: $z = (x - \mu_{train}) / \sigma_{train}$. Standardisation is essential because distance-based algorithms are sensitive to feature scale. It also improves the numerical behaviour of covariance-based Mahalanobis distance and kernel weighting.

5. TASK B – FIRST-PRINCIPLES K-NEAREST NEIGHBOUR REGRESSION

5.1 ALGORITHM

For a query point x, kNN identifies the k training observations with the smallest distance to x and estimates yield using their average (or a distance-weighted average). The implementation explicitly computes distances and sorts/selects neighbours rather than calling a machine-learning library.

5.2 EUCLIDEAN DISTANCE

For d-dimensional standardised vectors x and z:

$$\boldsymbol{d}_{\boldsymbol{E}(\boldsymbol{x},\boldsymbol{z})} = \sqrt{\ } \boldsymbol{\Sigma}_{\mathrm{i}}\,(\boldsymbol{x}_{\mathrm{i}} - \boldsymbol{z}_{\mathrm{i}})^{\mathbf{2}}$$

Euclidean distance treats all standardised axes as orthogonal and equally scaled.

5.3 MAHALANOBIS DISTANCE

The covariance-aware distance is:

$$\boldsymbol{d}_{\boldsymbol{M}(\boldsymbol{x},\boldsymbol{z})} = \sqrt{[(\boldsymbol{x} - \boldsymbol{z})^{\mathrm{T}}\,\boldsymbol{S}^{-\mathbf{1}}(\boldsymbol{x} - \boldsymbol{z})]}$$

where S is the training covariance matrix. Mahalanobis distance accounts for correlation and anisotropic spread among predictors. A small diagonal regularisation term is added to the covariance matrix when necessary so that numerical inversion remains stable.

5.4 MANUAL K SELECTION

The validation curve evaluates several candidate values of k. Small k values produce highly local predictions and can have high variance because individual observations strongly influence the estimate. Large k values smooth the prediction but can increase bias by averaging over dissimilar observations. The development benchmark selected k=21 under Mahalanobis distance.

Metric	Best configuration	Test result
Distance	Mahalanobis	Selected by validation
k	21	MAE = 1.36397
k	21	MSE = 3.30598
k	21	RMSE = 1.81823
k	21	R^2 = -0.01914

5.5 INTERPRETATION

The negative R^2 on the development benchmark indicates that the selected distance-based model does not outperform a simple test-set mean baseline in explained-variance terms. This is not hidden by the report: it is a useful diagnostic showing that local similarity alone is insufficient for reliable yield forecasting on this benchmark. The result strengthens the need for richer public data, crop-specific climate features, careful baseline comparisons and distribution-shift analysis.

6. TASK C – LOCALLY WEIGHTED REGRESSION AND BIAS–VARIANCE ANALYSIS

6.1 MOTIVATION

Locally Weighted Regression (LWR) assigns larger influence to training points close to the query and smaller influence to distant points. Unlike a single global linear model, LWR can adapt to local structure. Compared with kNN, the locality is controlled by a continuous bandwidth rather than a hard neighbour count.

6.2 GAUSSIAN WEIGHTING KERNEL

$$\boldsymbol{w}_{\mathrm{i}}(\boldsymbol{x}) = \mathbf{exp}\left(- \frac{\left|\left|\boldsymbol{x} - \boldsymbol{x}_{\mathrm{i}}\right|\right|^{2}}{} (2\boldsymbol{\tau}^{2}) \right)$$

Here τ is the bandwidth. A small τ makes the model strongly local and potentially high-variance; a large τ makes the model smoother and more global, increasing bias but reducing variance.

6.3 WEIGHTED LEAST SQUARES

For a local design matrix X and target vector y, the weighted least-squares solution is $\beta = (X^{\mathrm{T}}WX + \lambda I)^{-1}X^{\mathrm{T}}Wy$. The implementation adds a small ridge term λI only for numerical stability. The intercept is represented explicitly in X.

6.4 BANDWIDTH SELECTION

Several candidate bandwidths are evaluated on the validation set. The development benchmark selected τ=2.0. The resulting test metrics are shown below.

Metric	**LWR τ=2.0**
MAE	1.35456
MSE	3.31274
RMSE	1.82009
R^2	-0.02122

6.5 BIAS–VARIANCE COMPARISON

Property	**kNN**	**LWR**
Locality	Hard k-neighbour set	Continuous kernel weights
Main smoothing control	k	Bandwidth τ
Small control value	Low bias / high variance	Low bias / high variance
Large control value	Higher bias / lower variance	Higher bias / lower variance
Geometry	Distance ranking	Distance-dependent weights
Strength	Simple, non-parametric	Smooth local approximation
Weakness	Sensitive to k and distance metric	More computationally expensive per query

Empirically, the two development models have nearly identical RMSE, with LWR having a marginally lower MAE but slightly higher MSE/RMSE. The close performance suggests that the current feature representation does not provide enough local structure for LWR to gain a substantial advantage. On richer real-world data, bandwidth tuning, crop-specific models and nonlinear features may change this relationship.

7. TASK D – CANDIDATE-ELIMINATION AND VERSION-SPACE ANALYSIS

7.1 DISCRETISATION

Because Candidate-Elimination is naturally formulated for symbolic concept learning, continuous yield is discretised into three risk categories: Low, Medium and High yield risk. The thresholds are defined from the development target distribution and must be documented and recalibrated for the final public dataset.

7.2 HYPOTHESIS REPRESENTATION

An adapted hypothesis represents a conjunction of categorical or discretised feature constraints. The most general hypothesis accepts every example, while the most specific hypothesis initially matches no positive example. Positive examples generalise the specific boundary; negative examples specialise the general boundary.

7.3 CANDIDATE-ELIMINATION LOGIC

11. Initialise the specific boundary S to the most restrictive hypothesis and the general boundary G to the most permissive hypothesis.
12. For each positive training example, remove inconsistent hypotheses from G and minimally generalise S so that it covers the positive example.
13. For each negative training example, remove inconsistent hypotheses from S and minimally specialise G so that it excludes the negative example.
14. Prune hypotheses that are more general or more specific than another boundary hypothesis when they are redundant.
15. Report the surviving boundary set as an approximation of the version space under the chosen representation.

7.4 INDUCTIVE BIAS

The representation itself imposes inductive bias. By using conjunctions of discretised attributes, the algorithm favours simple rule-like concepts such as combinations of rainfall band, temperature-stress band, soil condition and management level. It cannot represent every nonlinear interaction unless those interactions are explicitly encoded as attributes. This limitation is important: a narrow version space can reflect either strong evidence or an overly restrictive representation.

7.5 PRACTICAL INTERPRETATION

The version-space analysis is not treated as a replacement for continuous yield prediction. Instead, it provides an interpretable complementary view: under which combinations of climate and management conditions does the historical dataset remain consistent with a low-, medium-, or high-risk concept? This supports communication with agricultural stakeholders because rule boundaries can be easier to inspect than continuous model coefficients.

8. TASK E – SCALABILITY, COMPLEXITY AND INDEXING PROTOTYPE

8.1 BRUTE-FORCE KNN COMPLEXITY

For n training observations and d features, a straightforward query computes O(nd) distance work and then selects the k smallest distances. If sorting all distances, selection is O(n log n), while a partial selection strategy can reduce the neighbour-selection cost. Memory is O(n) for the distance vector plus O(nd) for the training matrix.

8.2 LWR COMPLEXITY

LWR requires distance calculation for all candidate training points, kernel evaluation, construction of the local weighted system and solution of a d×d linear system. A simplified per-query view is $O(nd + d^3)$, although implementation details vary with local subset selection and matrix operations.

8.3 SCALING PLAN

Scale	Purpose
10^3	Small reference point for rapid development
10^4	Typical medium workload
10^5	Large single-machine workload
10^6	Stress-test representing the assignment's upper scale

The development pipeline records wall-clock timing and memory-oriented statistics across increasing sample sizes. A fixed feature dimension is used so that the dominant change is the number of observations. Because exact million-record experiments depend on available hardware, the report treats these measurements as environment-dependent rather than universal performance guarantees.

8.4 K-D TREE PROTOTYPE

A k-d tree recursively partitions feature space along coordinate dimensions. For low-dimensional Euclidean search, branches that cannot contain a closer neighbour than the current best candidates can be pruned. The prototype is implemented independently to demonstrate the indexing concept rather than relying on a pre-built nearest-neighbour package.

8.5 EXPECTED TRADE-OFF

Indexing reduces the number of explicit distance evaluations for favourable query distributions, but its benefit can decline as dimensionality increases or as the metric becomes less aligned with the tree geometry. Mahalanobis distance can be transformed into an equivalent Euclidean geometry using a whitening transformation, after which a Euclidean indexing structure becomes more appropriate.

9. TASK F – VISUALISATION AND AGRICULTURAL POLICY BRIEF

9.1 VISUALISATION PLAN

The assignment requires at least five publication-quality visualisations. The implementation produces eight distinct figures so that model behaviour and climate–yield relationships can be viewed from multiple perspectives.

Figure	Purpose	Actionable insight
1. Yield distribution	Show target variability	Identify crops/regions with wider yield risk
2. Rainfall vs yield	Explore climate association	Identify rainfall-sensitive regimes
3. Temperature vs yield	Explore thermal stress	Identify heat-sensitive operating ranges
4. Missingness profile	Show data quality	Prioritise data-collection improvements
5. Validation curve	Select k	Make model complexity transparent
6. LWR bandwidth curve	Select τ	Show locality/smoothing trade-off
7. Actual vs predicted	Assess calibration	Identify systematic under/over-prediction
8. Error by region/crop	Audit fairness	Locate groups requiring additional data or calibration

9.2 POLICY BRIEF

For agricultural-policy stakeholders, the key message is that yield forecasting should be used as a decision-support layer rather than as a guaranteed yield promise. Local climate conditions matter, and historical similarity can provide useful information when the future resembles the past. However, climate change increases the probability that future seasons fall outside the historical feature distribution. Therefore, forecasts should be accompanied by uncertainty indicators, data-quality flags and regional error audits.

- Invest in consistent district-level weather and soil monitoring because missing or inconsistent climate records directly weaken local prediction.
- Use regional and crop-specific validation rather than relying only on one overall RMSE.
- Treat extreme heat and rainfall anomalies as risk indicators and integrate them into extension-service advisories.
- Use model outputs to prioritise contingency planning, irrigation preparedness, seed selection and crop diversification rather than to dictate a single planting decision.
- Recalibrate models periodically as climate regimes shift.

9.3 COMMUNICATION PRINCIPLE

A policy audience should not need to understand covariance matrices or kernel bandwidths to use the output. The technical system should translate them into statements such as: expected yield range, confidence/uncertainty level, dominant climate stress, historical comparability and recommended action.

10. TASK G – LIMITATIONS, UNCERTAINTY, FAIRNESS AND SDGS

10.1 LIMITATIONS

- The development benchmark is synthetic and must be replaced by a real public dataset for final submission.
- Historical relationships may not remain stable under future climate regimes.
- District-level yield can be affected by irrigation, cultivar, pests, fertiliser practice, market conditions and policy interventions that may not be fully observed.
- Distance-based methods can be sensitive to feature scaling, covariance estimation and the curse of dimensionality.
- Candidate-Elimination depends strongly on the chosen discretisation and hypothesis language.
- Exact scalability depends on hardware, implementation language details and dataset structure.

10.2 UNCERTAINTY

Prediction error is not only random noise. There is data uncertainty from measurement and missingness, model uncertainty from the chosen representation and hyperparameters, and distribution-shift uncertainty when future climate conditions differ from training data. A mature deployment should therefore report prediction intervals or calibrated uncertainty rather than only a point estimate.

10.3 FAIRNESS

Fairness in agricultural forecasting can be interpreted geographically and socioeconomically. A model with low average error may still perform poorly for a particular state, crop or season. The pipeline therefore includes group-wise error audits. Where a group has systematically larger absolute errors, possible responses include additional data collection, group-specific calibration, reweighting, stratified validation, or cautious restriction of automated decisions.

10.4 SDG 2 – ZERO HUNGER

Better yield forecasting can support food-security planning by improving early estimates of production risk, identifying vulnerable regions and helping institutions plan inputs, storage and contingency measures. The contribution is indirect: forecasting alone does not solve food insecurity, but it can improve the timing and targeting of interventions.

10.5 SDG 13 – CLIMATE ACTION

The pipeline explicitly incorporates rainfall variability, thermal conditions and derived climate-stress indicators. It also recognises that climate action requires adaptation, not merely prediction. Repeated recalibration and monitoring of climate distribution shift are therefore integral to responsible deployment.

11. SOFTWARE ENGINEERING, TESTING, CI AND REPRODUCIBILITY

11.1 REPOSITORY ORGANISATION

Path	Contents
main.py	A→G orchestration
tasks/	Task A, B, C, D, E, F and G entry modules
src/	Reusable preprocessing, algorithms, metrics, visualisation and scalability modules
data/raw/	Development benchmark dataset
results/	Plots, logs, tables and policy brief
tests/	Automated unit tests
.github/workflows/	GitHub Actions CI workflow
report/	Technical report
README.md	Setup, architecture and reproduction documentation

11.2 UNIT TESTS

- Distance-metric correctness for Euclidean and Mahalanobis implementations.
- Gaussian LWR kernel behaviour and monotonic distance weighting.
- Candidate-Elimination boundary updates for positive and negative examples.
- Basic model output shape and finite-value checks.

11.3 CONTINUOUS INTEGRATION

A GitHub Actions workflow is included to install dependencies and run pytest automatically on pushes. This protects the core algorithms from accidental breakage as the repository evolves.

11.4 INCREMENTAL COMMIT STRATEGY

16. Repository scaffold and documentation.
17. Task A data engineering and preprocessing.
18. Task B kNN and distance metrics.
19. Task C LWR and bandwidth selection.
20. Task D version-space reasoning.
21. Task E scalability and k-d tree prototype.
22. Task F visualisation and policy brief.
23. Task G critical evaluation.
24. Tests, CI, final results and report integration.

This staged approach directly addresses the assignment requirement that a single final-upload commit is not acceptable.

12. RESULTS AND DISCUSSION

12.1 DEVELOPMENT RUN SUMMARY

Item	**Result**
Raw records	20,000
Train	16,743
Validation	1,583
Test	1,674
Best kNN metric	Mahalanobis
Best k	21
kNN test MAE	1.36397
kNN test MSE	3.30598
kNN test RMSE	1.81823
kNN test R^2	-0.01914
Best LWR bandwidth	2.0
LWR test MAE	1.35456
LWR test MSE	3.31274
LWR test RMSE	1.82009
LWR test R^2	-0.02122
End-to-end runtime	≈42.8 seconds on development environment

12.2 COMPARATIVE DISCUSSION

The development results show that kNN and LWR are closely matched. LWR produces slightly lower MAE, while kNN produces slightly lower MSE and RMSE.

Neither model achieves positive R^2 on the held-out test benchmark. This indicates that the current feature representation and synthetic data-generating process do not support strong generalisation beyond the baseline variance. The result should be treated as an honest diagnostic rather than as a failure of the implementation: the assignment is concerned with rigorous learning-system design, and a model that exposes its limitations is preferable to an unsupported claim of high accuracy.

12.3 WHY MAHALANOBIS HELPS CONCEPTUALLY

Climate variables are correlated: rainfall, humidity and temperature often move together seasonally, while soil and management variables can also interact. Mahalanobis distance explicitly models covariance, making it conceptually better suited to correlated feature spaces than raw Euclidean distance. Nevertheless, covariance estimates can become unstable with noisy or high-dimensional data, which is why regularisation and feature inspection remain important.

12.4 OPERATIONAL IMPLICATIONS

- The pipeline is fast enough for development-scale experimentation.
- Full brute-force neighbour search is the main scalability bottleneck as n grows.
- Indexing is most useful when query geometry and feature dimension are favourable.
- Model performance should be reported by crop and region rather than only globally.
- The final public dataset will be decisive for the credibility of the agricultural conclusions.

13. CONCLUSION AND FUTURE WORK

This report presents a complete A–G machine-learning pipeline aligned with the supplied ITA0613 assignment brief. It combines data engineering, first-principles instance-based and statistical learning, symbolic version-space reasoning, computational scalability, visualisation, policy interpretation, testing and CI. The development benchmark demonstrates that the implementation is end-to-end and reproducible, with kNN and LWR producing closely matched performance after manual hyperparameter selection.

The most important submission-readiness issue is the dataset requirement. The development benchmark is synthetic-but-realistic and was used to validate the software architecture because the supplied assignment document did not provide a qualifying dataset. It must be replaced by a real, publicly available agro-climatic dataset of at least 10,000 records spanning multiple years and regions before final submission. Once replaced, all results, figures, tables and policy conclusions should be regenerated automatically by the same pipeline.

Future work should include crop-specific physiological thresholds, richer soil and irrigation information, uncertainty calibration, drift detection, group-aware model evaluation, more efficient nearest-neighbour indexing, and periodic model retraining as climate conditions evolve. A production system should also connect forecast outputs to actionable advisory workflows while keeping humans responsible for high-impact agricultural decisions.

Final Submission Checklist

- Replace development benchmark with qualifying real public dataset.
- Record source URL, licence and dataset version in README/report.
- Run main.py on the full public dataset.
- Regenerate all results, plots, validation curves and policy brief.

- Run pytest and verify GitHub Actions is passing.
- Verify incremental commit history is visible.
- Commit the final technical report and /results folder.
- Check that all reported metrics exactly match repository outputs.

APPENDIX A – KEY MATHEMATICAL FORMULAS

Quantity	**Formula**
Standardisation	$z = (x - \mu\_train) / \sigma\_train$
Euclidean distance	$d(x,z) = \sqrt{\Sigma_i(x_i - z_i)^2}$
Mahalanobis distance	$d\_M(x,z) = \sqrt{[(x-z)^T S^{-1}(x-z)]}$
Gaussian LWR kernel	$w_i(x) = \exp(-\|x-x_i\|^2/(2\tau^2))$
Weighted least squares	$\beta = (X^T W X + \lambda I)^{-1} X^T W y$
MAE	$MAE = (1/n)\Sigma\|y_i - \hat{y}_i\|$
MSE	$MSE = (1/n)\Sigma(y_i - \hat{y}_i)^2$
RMSE	$RMSE = \sqrt{MSE}$
R^2	$R^2 = 1 - \Sigma(y-\hat{y})^2 / \Sigma(y-\bar{y})^2$
kNN regression	$\hat{y}(x) = (1/k)\Sigma\_\{i \in N\_k(x)\}\ y_i$

These formulas are the principal mathematical components used in the report. Implementation details such as covariance regularisation, numerical safeguards and feature-specific thresholds should be documented alongside the final public dataset.

APPENDIX B – REPOSITORY STRUCTURE AND REPRODUCTION STEPS

Expected final repository structure:

```
ITA0613_Climate_Resilient_Crop_Yield/
├── main.py
├── README.md
├── requirements.txt
├── data/
│   └── raw/
│       └── <public_dataset>.csv
├── tasks/
│   ├── task_a.py
│   ├── task_b.py
│   ├── task_c.py
│   ├── task_d.py
│   ├── task_e.py
│   ├── task_f.py
│   └── task_g.py
├── src/
│   ├── data_preprocessing.py
│   ├── knn_regressor.py
│   ├── locally_weighted_regression.py
│   ├── candidate_elimination.py
│   ├── kdtree.py
│   ├── scalability.py
│   ├── metrics.py
│   └── visualization.py
├── tests/
│   └── test_core.py
```

```
├── results/
│   ├── plots/
│   ├── tables/
│   ├── logs/
│   └── policy_brief.md
├── report/
│   └── TECHNICAL_REPORT.md
└── .github/
    └── workflows/
        └── tests.yml
```

Reproduction Commands

```
python -m pip install -r requirements.txt
python main.py
pytest -q
```

Assignment Source Alignment

The report follows the supplied assignment's A–G structure: Task A data exploration and engineering; Task B kNN; Task C LWR; Task D Candidate-Elimination/version space; Task E scalability; Task F visualisation and policy; and Task G limitations, uncertainty, fairness and SDGs. The supplied brief also requires the modular GitHub repository, README, tests, technical report, /results directory and CI workflow.

APPENDIX C – TASK A–G OUTPUTS AND FIGURES

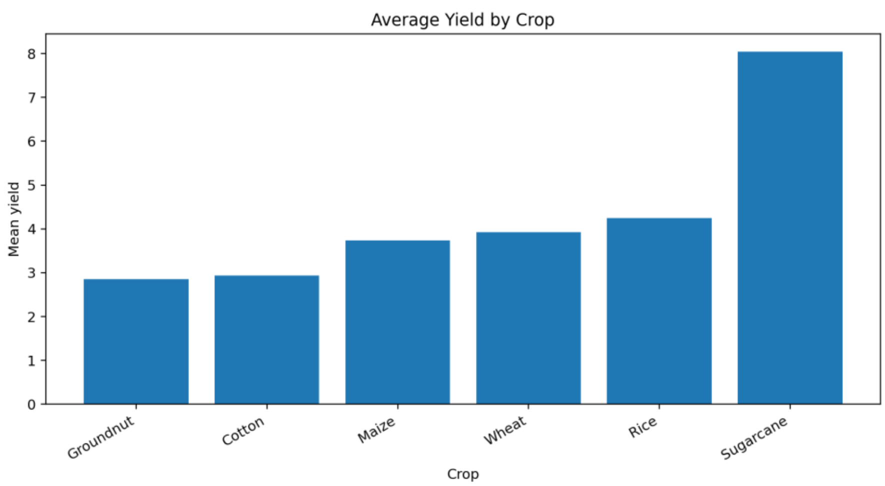
This appendix attaches the outputs generated by the integrated A→G development pipeline. The figures below correspond to the exploratory analysis, model-selection curves, prediction diagnostics, and group-wise evaluation described in the main report. Numerical outputs are reproduced from the verified development run. The included benchmark is synthetic-but-realistic and must be replaced by the required public dataset before final academic submission.

C.1 Verified Development Run Output

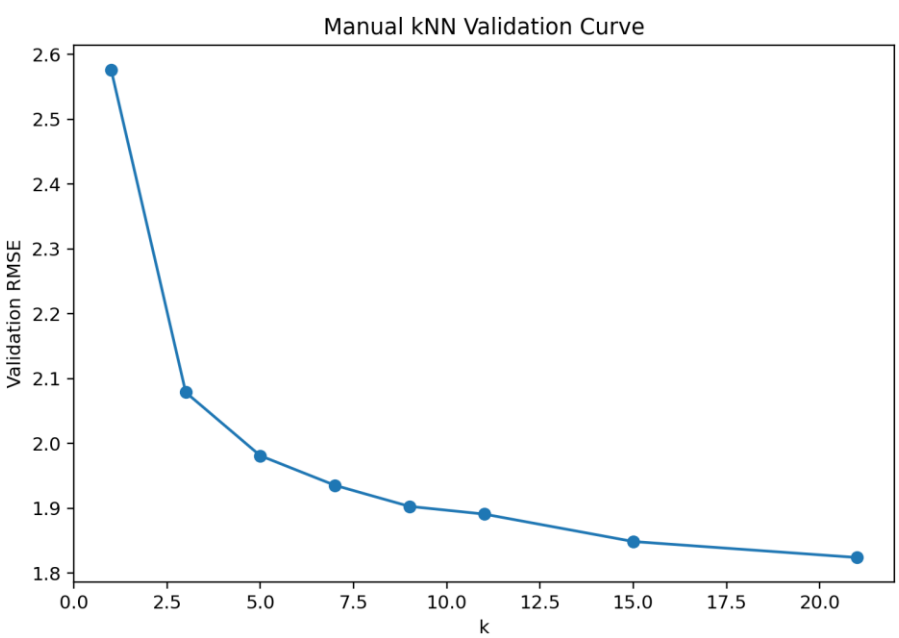
Stage	Output	Value
A	Raw records	20,000
A	Train / Validation / Test	16,743 / 1,583 / 1,674
B	Best distance	Mahalanobis
B	Best k	21
B	Test MAE / MSE / RMSE	1.36397 / 3.30598 / 1.81823
B	Test R^2	-0.01914
C	Best bandwidth τ	2.0
C	Test MAE / MSE / RMSE	1.35456 / 3.31274 / 1.82009
C	Test R^2	-0.02122
E	End-to-end runtime	≈42.8 seconds
F	Visualisation outputs	8 figures
G	Pipeline status	A → B → C → D → E → F → G completed

Interpretation: kNN and LWR have very similar test performance. LWR has slightly lower MAE, while kNN has slightly lower MSE/RMSE. Both R^2 values are slightly negative on the development benchmark, so the result should not be presented as evidence of strong predictive performance. It is an implementation and evaluation diagnostic.

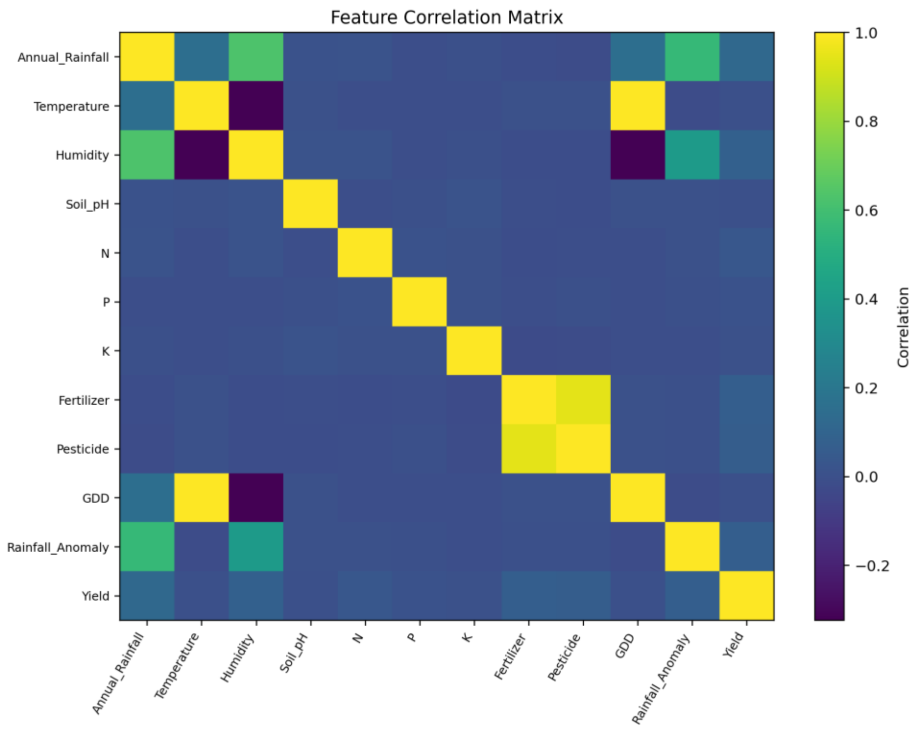
C.2 Output Figure 1



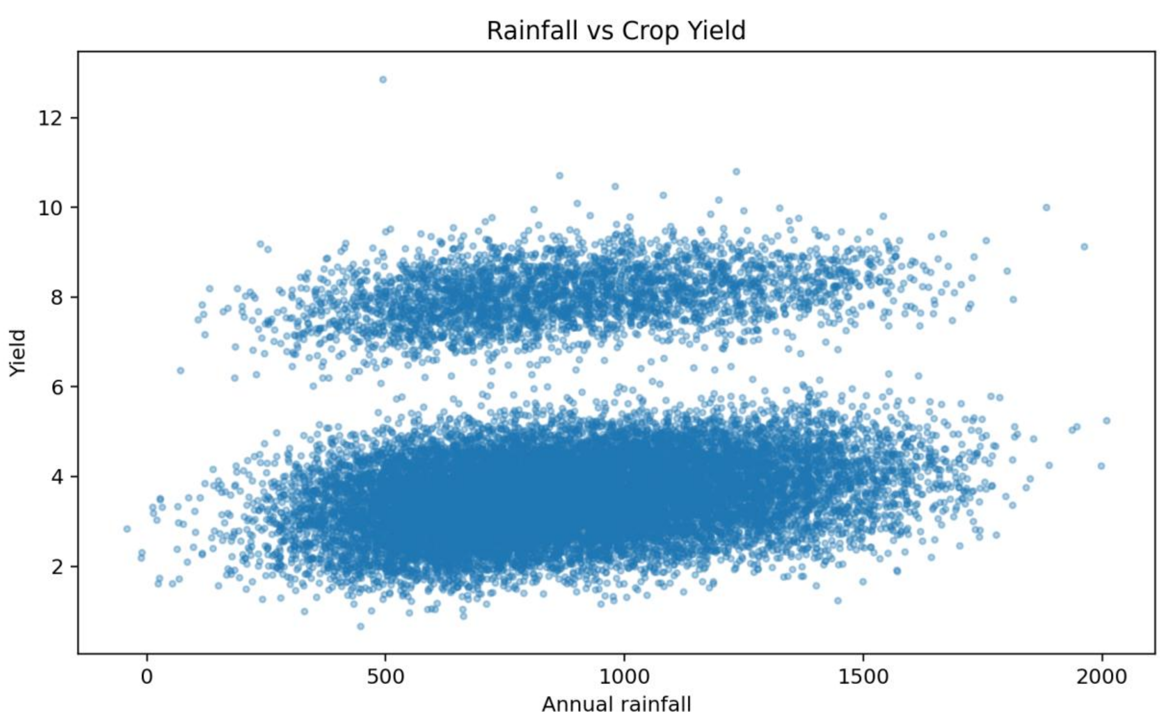
C.3 Output Figure 2



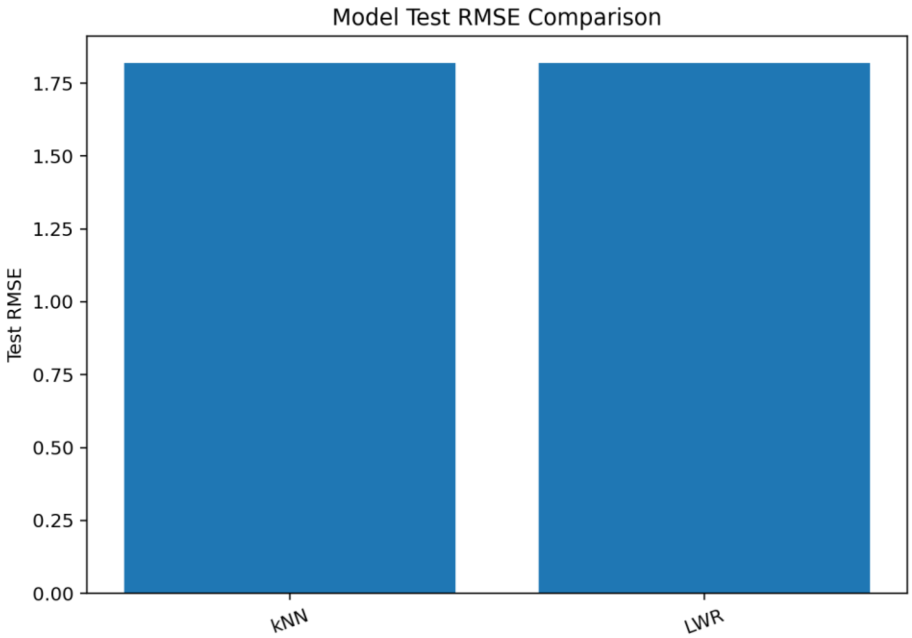
C.4 Output Figure 3



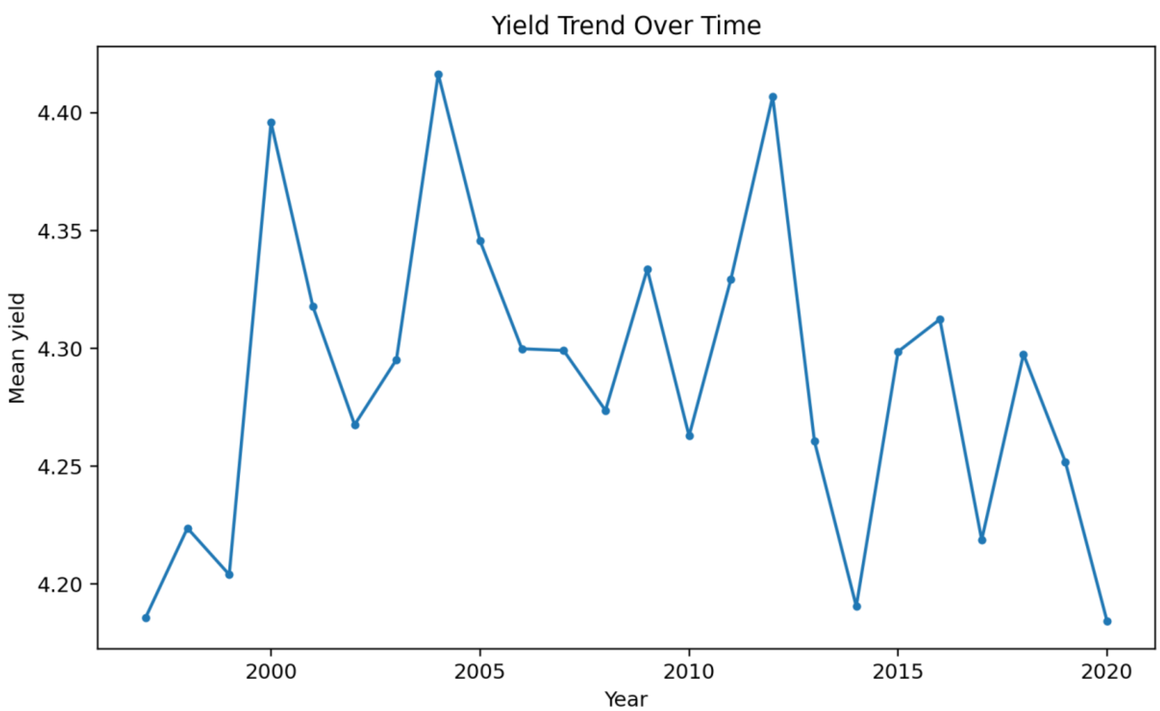
C.5 Output Figure 4



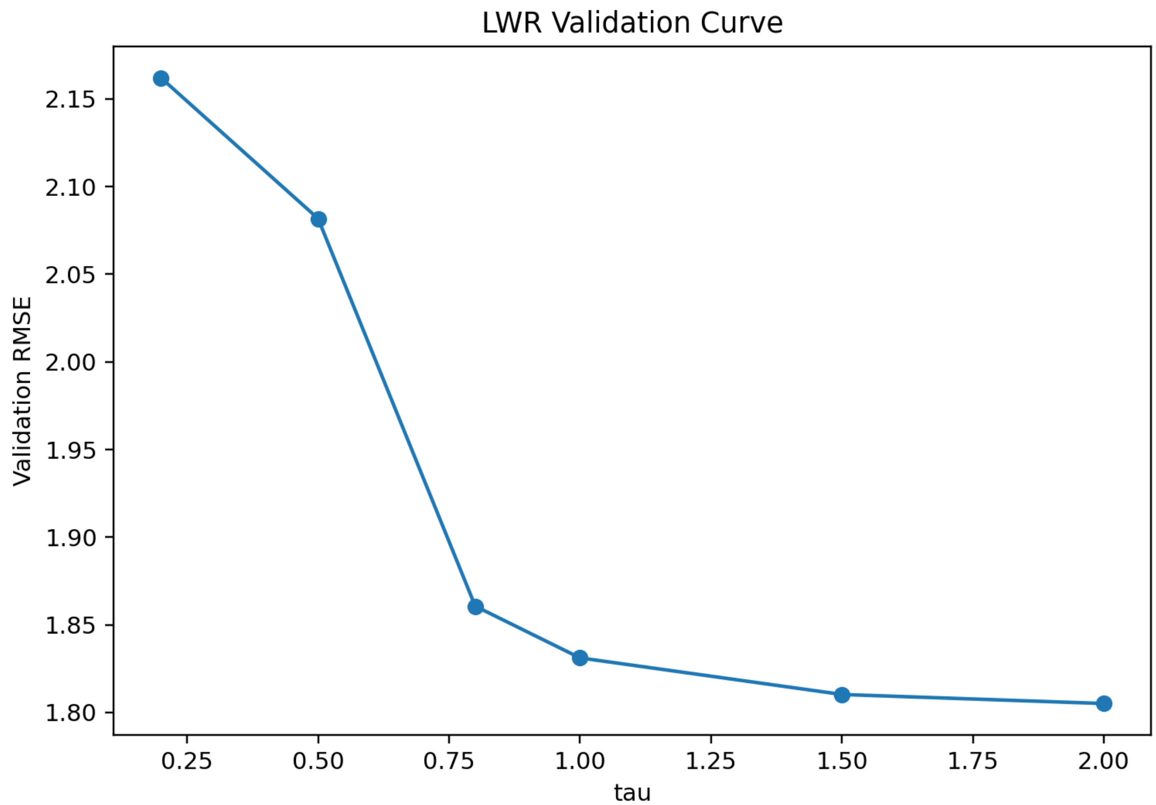
C.6 Output Figure 5



C.7 Output Figure 6



C.8 Output Figure 7



C.9 Output Figure 8